Tecnológico de Software

Asignatura:

Programación Estructurada

Proyecto Final Unidad III.

Grado y Grupo:

1°C

Carrera:

TSU. Desarrollo de Software y Negocios Digitales

Alumno:

Joaquín Uriona

Astrit Airan Cetzal Cetzal

Carlos Llanes Rodriguez

Jesús Omar Uc Domínguez

Nuestro juego consiste en un laberinto cuyos muros son “#” donde el usuario debe encontrar la salida y para esto primero debe:

1. Escribir 1 para jugar
2. Ingresar su nombre, porque sin este el programa no lo llevara al laberinto
3. Escribir las letras A, W, S, D para moverse
4. Si decide salir antes de terminar ingresar Q

Diseño del programa

Arreglos unidimensionales:

Las direcciones que se le muestran al usuario en pantalla se guardan dentro de un arreglo tipo String para luego mostrarlas en orden de lista al usuario.

Con un arreglo unidimensional tipo int se almacenan las filas y las columnas para encontrar la entrada.

Para calcular las nuevas posiciones del usuario conforme se va moviendo dentro del laberinto se utiliza un vector tipo int.

Arreglos bidimensionales:

El archivo “Laberinto.txt” se lee y se cuenta el número de filas y columnas y se guardan dentro de un arreglo tipo char.

Archivos:

En un archivo laberinto.txt tenemos la matriz laberinto y en un archivo “laberinto_historial.txt” se guardan los resultados del juego.

Métodos principales y su función

cargarLaberinto(): lee la información del archivo y los almacena en un arreglo bidimensional. Muestra el número de filas y columnas de la matriz.

mostrarDirecciones(): muestra las direcciones disponibles a las que puede acceder el usuario para moverse dentro del laberinto.

cargarLaberinto(): lee el archivo .txt para contar el número de filas y columnas de la matriz, para los resultados.

mostrarLaberinto(): muestra el laberinto original y conforme el usuario se va moviendo a través de este, lo que hace es actualizarse y mostrarse.

encontrarEntrada(): encuentra la entrada designada como “E”, en la matriz, si esta no se encuentra entonces no se podrá interactuar con el laberinto.

calcularNuevaPosicion(): calcula la nueva posición del usuario dentro de la matriz y se actualiza el laberinto.

estaSalida(): si la fila y la columna se encuentran en la letra S entonces se muestra el laberinto.

guardarResultados(): guarda el nombre del jugador, el número de movimiento que ha realizado y si llegó o no a la salida.

Funciones predefinidas, lectura y escritura de archivos, validaciones:

```java
public static boolean validarNombre(String nombre) {
    if (nombre == null || nombre.trim().isEmpty()) {
        System.out.println(x: "Error: El nombre no puede estar vacío.");
        return false;
    }

    for (int i = 0; i < nombre.length(); i++) {
        char c = nombre.charAt(i);
        if (!Character.isLetter(c) && c != ' ') {
            System.out.println(x: "Error: El nombre solo puede contener letras y espacios.");
            return false;
        }
    }

    if (nombre.trim().length() < 2) {
        System.out.println(x: "Error: El nombre debe tener al menos 2 caracteres.");
        return false;
    }

    return true;
}
```

```java
public static boolean validarOpcionMenu(int opcion, int min, int max) {
    if (opcion < min || opcion > max) {
        System.out.println("Error: Opción no válida. Debe estar entre " + min + " y " + max + ".");
        return false;
    }
    return true;
}
```

```java
public static boolean validarMovimiento(String movimiento) {
    if (movimiento == null || movimiento.isEmpty()) {
        System.out.println(x: "Error: Debe ingresar un movimiento.");
        return false;
    }

    String mov = movimiento.toUpperCase().trim();

    if (mov.length() != 1) {
        System.out.println(x: "Error: El movimiento debe ser una sola letra.");
        return false;
    }

    char comando = mov.charAt(index: 0);
    if (comando != 'W' && comando != 'A' && comando != 'S' && comando != 'D') {
        System.out.println(x: "Error: Movimiento inválido. Use W (arriba), A (izquierda), S (abajo), D (derecha).");
        return false;
    }

    return true;
}
```

```java
public static boolean validarCoordenadas(int fila, int columna, char[][] laberinto) {
    if (fila < 0 || fila >= laberinto.length) {
        System.out.println(x: "Error: No puedes salir de los límites del laberinto (fila fuera de rango).");
        return false;
    }

    if (columna < 0 || columna >= laberinto[0].length) {
        System.out.println(x: "Error: No puedes salir de los límites del laberinto (columna fuera de rango).");
        return false;
    }

    return true;
}
```

```java
public static boolean validarNoMuro(int fila, int columna, char[][] laberinto) {
    if (laberinto[fila][columna] == '#') {
        System.out.println(x: "Error: No puedes atravesar un muro.");
        return false;
    }
    return true;
}
```

```java
public static boolean validarMovimientoPosible(int fila, int columna, char[][] laberinto) {
    if (!validarCoordenadas(fila, columna, laberinto)) {
        return false;
    }

    if (!validarNoMuro(fila, columna, laberinto)) {
        return false;
    }

    return true;
}
```

```java
public static boolean validarLaberinto(char[][] laberinto) {
    if (laberinto == null || laberinto.length == 0) {
        System.out.println(x: "Error: El laberinto está vacío o no se cargó correctamente.");
        return false;
    }

    boolean tieneEntrada = false;
    boolean tieneSalida = false;

    for (int i = 0; i < laberinto.length; i++) {
        for (int j = 0; j < laberinto[i].length; j++) {
            if (laberinto[i][j] == 'E') {
                tieneEntrada = true;
            }
            if (laberinto[i][j] == 'S') {
                tieneSalida = true;
            }
        }
    }

    if (!tieneEntrada) {
        System.out.println(x: "Error: El laberinto no tiene una entrada (E).");
        return false;
    }

    if (!tieneSalida) {
        System.out.println(x: "Error: El laberinto no tiene una salida (S).");
        return false;
    }

    return true;
}
```

```java
public static boolean validarArchivoExiste(String nombreArchivo) {
    try {
        BufferedReader br = new BufferedReader(new FileReader(nombreArchivo));
        br.close();
        return true;
    } catch (Exception e) {
        System.out.println("Error: No se pudo encontrar o leer el archivo '" + nombreArchivo + "'.");
        return false;
    }
}
```

```java
static void guardarResultado(String nombreMayus, int movimientos, boolean llego, String archivo) {
    try (PrintWriter pw = new PrintWriter(new FileWriter(archivo, append: true))) {
        pw.println(nombreMayus + " | movimientos: " + movimientos + " | llego: " + llego);
    } catch (IOException e) {
        System.out.println("Error al guardar resultados: " + e.getMessage());
    }
}
```

```java
    String nombre;
do {
System.out.print(s: "Ingresa tu nombre: ");
nombre = sc.nextLine().trim();
if (!validarNombre(nombre)) {
System.out.println(x: "Vuelva a intentarlo");
System.out.println(x: "");
}
} while (!validarNombre(nombre));
String nombreMayus = nombre.toUpperCase();
```

Capturas de pantalla de VSCode

```
===== MENU DEL JUEGO =====
1. Jugar
2. Salir
1
Ingresa tu nombre: Airan Cetzal
Laberinto cargado exitosamente (7 filas x 14 columnas)

===== LABERINTO =====
##############
#            #
## ### ##### #
#    # #     #
#    # #  ####
#    # #     S
######P#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================

Usa W, A, S, D para moverte. Presiona 'Q' para salir.
```

```
===== LABERINTO =====
##############
#            #
## ### ##### #
#    # #     #
#    # #  ####
#    #P#     S
######E#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================

Usa W, A, S, D para moverte. Presiona 'Q' para salir.
w

===== LABERINTO =====
##############
#            #
## ### ##### #
#    # #     #
#    #P#  ####
#    # #     S
######E#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================
```

```
===== LABERINTO =====
##############
#            #
## ### ##### #
#    # # P   #
#    # #  ####
#    # #     S
######E#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================

Usa W, A, S, D para moverte. Presiona 'Q' para salir.
```

```
===== LABERINTO =====
##############
#            #
## ### ##### #
#    # #     #
#    # #  ####
#    # #     P
######E#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================

¡Felicidades! Llegaste a la salida (S)!
Resultado guardado en laberinto_historial.txt
```

laberinto_historial.txt × | Laberinto1.txt | LaberintoAct.java

laberinto_historial.txt

```
JOAQUIN | movimientos: 22 | llego: true
JOAQUINNNN | movimientos: 22 | llego: true
JOAUIN | movimientos: 21 | llego: false
ASTRIT | movimientos: 4 | llego: false
SD | movimientos: 22 | llego: true
LK | movimientos: 0 | llego: false
AIRAN CETZAL | movimientos: 26 | llego: true
```

Capturas de la validación de errores

```
===== MENU DEL JUEGO =====
1. Jugar
2. Salir
a
Error general: For input string: "a"

===== MENU DEL JUEGO =====
1. Jugar
2. Salir
3
Opción no válida. Intente nuevamente.

===== MENU DEL JUEGO =====
1. Jugar
2. Salir
1
Ingresa tu nombre:
Error: El nombre no puede estar vacío.
Vuelva a intentarlo

Error: El nombre no puede estar vacío.
Ingresa tu nombre: -
Error: El nombre solo puede contener letras y espacios.
Vuelva a intentarlo

Error: El nombre solo puede contener letras y espacios.
Ingresa tu nombre:
Error: El nombre no puede estar vacío.
Vuelva a intentarlo

Error: El nombre no puede estar vacío.
Ingresa tu nombre: a
Error: El nombre debe tener al menos 2 caracteres.
Vuelva a intentarlo

Error: El nombre debe tener al menos 2 caracteres.
Ingresa tu nombre:
```

```
Laberinto cargado exitosamente (7 filas x 14 columnas)

===== LABERINTO =====
##############
#            #
## ### ##### #
#    # #     #
#    # #  ####
#    # #     S
######P#######
====================


===== DIRECCIONES DISPONIBLES =====
1. W - Arriba
2. A - Izquierda
3. S - Abajo
4. D - Derecha
==================================

Usa W, A, S, D para moverte. Presiona 'Q' para salir.
a
Error: No puedes atravesar un muro.
Usa W, A, S, D para moverte. Presiona 'Q' para salir.
m
Comando inválido. Usa W, A, S, D o Q para salir.
Usa W, A, S, D para moverte. Presiona 'Q' para salir.
-
Comando inválido. Usa W, A, S, D o Q para salir.
Usa W, A, S, D para moverte. Presiona 'Q' para salir.
```

Conclusiones

Astrit: Aprendí mucho al hacer este proyecto, a cómo emplear los arreglos unidimensionales, a validar errores, a usar los archivos, en pocas palabras apliqué lo visto previamente en clase. Lo más difícil fue que se agregaran las funciones porque fue lo que llevó hora en hacerse, y pues gracias a que se puede investigar encontramos la solución igual al principio no entendía cómo debía de ser la estructura del código pero gracias a mis compañeros lo logré. Si tuviéramos más tiempo el menú tendría más opciones de laberintos para jugar y no solo una opción.

Joaquin: Durante el proceso de escribir el codigo pude ver realmente cómo se trabajaba en un equipo de programadores cuando el proyecto es largo y creo que me ayudó a mejorar como líder y diseñador del plan de ejecución del mismo, la parte más difícil fue implementar todas las funciones y validaciones necesarias y interconectarlas para no duplicar código, que fuera eficiente y funcional. La parte de mover al personaje me pareció complicada porque a pesar de que sabia que tenía que aumentar y disminuir x e y, al principio realice una programación muy compleja cuando era básica.

Si tuviera más tiempo para desarrollar el juego, yo mejoraría la interfaz del juego y agregaría más laberintos para que sea más largo y también implementaría una función para ver que usuarios lo hacen más rápido, en cuanto tiempo, etc.

Carlos: Con la realización de este proyecto pude comprender como aplicar de manera complementaria todos los contenidos vistos a lo largo de la materia de programación estructurada así como lo importante que es usar validaciones para evitar que nuestros programas se cierren ante cualquier error. Lo que más difícil se me hizo fue la parte de cómo hacer que el jugador se mueva dentro del laberinto, que aunque no me correspondió hacerlo a mí sí me causó mucha duda como desarrollarlo. Si tuviera más

tiempo podría agregar más archivos de tipo .txt para generar más laberintos y así más niveles.

Jesús: Durante el desarrollo de este proyecto aprendí a trabajar de manera más organizada dentro del código y a comprender mejor cómo se conectan las funciones entre sí para que el programa sea completamente funcional. También reforcé el uso de arreglos y la importancia de validar cada dato que ingresa el usuario para evitar errores inesperados. Lo más complicado para mí fue asegurarme de que cada parte del menú respondiera correctamente y que no hubiera fallos al leer los archivos del laberinto, ya que cualquier detalle afectaba el funcionamiento completo. Si tuviera más tiempo para seguir mejorando el proyecto, implementaría nuevos tipos de laberintos y agregaría niveles de dificultad.

Referencias

Franco Martínez, E. A. (s. f.). *Tema 12: Arreglos estáticos en C*. Docencia Eafranco. Recuperado de https://docencia.eafranco.com/materiales/algoritmia/12/Tema12.pdf

Solución Ingenieril. (2025). *Arreglos unidimensionales o vectores – Programación en C*. SoluciónIngenieril.com. Recuperado de https://solucioningenieril.com/programacion_en_c/arreglos_unidimensionales_o_vectores

W3SchoolsES. (s. f.). *C Leer archivos*. W3Schools — versión en español. Recuperado de https://w3schoolsua.github.io/c/c_files_read_es.html

LabEx. (s. f.). *Manejo de errores al abrir archivos en C*. LabEx. Recuperado de https://labex.io/es/tutorials/c-how-to-handle-file-open-errors-431171